RAJALAKSHMI ENGINEERING COLLEGE

R Ashwin Kumar 251801033-2025-AIDS

R2

| | |
|---|---|
| **Started on** | Thursday, 8 October 2026, 10:05 AM |
| **State** | Finished |
| **Completed on** | Thursday, 8 October 2026, 10:07 AM |
| **Time taken** | 2 mins 9 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00

**Playing with Chessboard:**

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:**

**Input**

3

**1** 2 4

**2** 3 4

**8 7 1**

**Output:**

19

**Explanation:**

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

**Input Format**

First Line contains the integer n

The next n lines contain the n*n chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;

    scanf("%d", &n);

    int board[n][n];
    int dp[n][n];

    // Read the chessboard
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &board[i][j]);
        }
    }

    // Starting cell
    dp[0][0] = board[0][0];

    // First row: can only come from the left
    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + board[0][j];
    }

    // First column: can only come from above
    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + board[i][0];
    }

    // Fill the remaining cells
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {

            if (dp[i - 1][j] > dp[i][j - 1]) {
                dp[i][j] = board[i][j] + dp[i - 1][j];
```

```
            } else {
                dp[i][j] = board[i][j] + dp[i][j - 1];
            }
        }
    }

    // Maximum monetary value
    printf("%d\n", dp[n - 1][n - 1]);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 2 4<br>2 3 4<br>8 7 1 | 19 | 19 | ✔ |
| ✔ | 3<br>1 3 1<br>1 5 1<br>4 2 1 | 12 | 12 | ✔ |
| ✔ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28 | 28 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course